

VIT BHOPAL UNIVERSITY
School of Computing Science and Engineering
Course: Programming in Java

PROJECT REPORT
ON
“MEDICINE TRACKING SYSTEM”

Submitted in partial fulfillment of the academic requirements for the course

Student Name:	Tanmay Sharda
Registration No.:	25BAI11394
Branch / Specialization:	B.Tech CSE (AIML)
Faculty Mentor:	Dr. Sanat Jain
Academic Year:	2025 – 2026

1. Cover Page Details

- **Project Title:** Medicine Tracking System
- **Candidate Name:** Tanmay Sharda
- **Registration Number:** 25BAI11394
- **Program:** B.Tech Computer Science and Engineering (AI & ML)
- **Course Name:** Programming in Java
- **Faculty Guide:** Dr. Sanat Jain
- **Institution:** VIT Bhopal University

2. Introduction

The **Medicine Tracking System** is a modular, console-based stock management application developed in Java. It is designed to assist small pharmacies, campus health centers, and dispensary units in recording, managing, and tracking their pharmaceutical inventory. The project demonstrates core Computer Science principles and fundamental Java programming competencies, including Object-Oriented Design (abstraction, encapsulation, and modularity), the Java Collections framework (`HashMap`), custom Exception Handling, robust Input Validation, and persistent File Input/Output (I/O) mechanisms.

3. Problem Statement

In small dispensaries and clinics, inventory tracking is commonly performed manually using paper registers or static spreadsheets. This manual workflow is highly error-prone, inefficient, lacks real-time verification, and easily results in critical medicine stockouts or unnoticed expirations.

Proposed Solution

The Medicine Tracking System provides an automated, menu-driven command-line system that maintains accurate inventory levels, automates stock replenishment and deduction, validates user inputs against data-entry mistakes, issues proactive low-stock warnings, and ensures data persists across sessions via file storage.

4. Functional Requirements

The project satisfies the requirement of having distinct, robust functional modules:

1. Module 1: Medicine Registration (Add Item)

- Allows the user to register a new medicine with name, initial batch quantity, and expiry date (MM/YYYY).
- Validates that duplicate medicine names are prevented and initial quantity is non-negative.

2. Module 2: Stock Management (Update Stock)

- Allows incrementing stock when fresh supplies arrive (restocking).
- Allows decrementing stock when medicines are dispensed to patients.
- Prevents negative inventory balance using custom exceptions.

3. Module 3: Inventory Monitoring & Reporting (View Records)

- Displays all registered medicines in a clear tabular format with current stock and expiry dates.

4. Module 4: Low-Stock Alert System

- Automatically inspects current quantities against a safety threshold (10 units) and highlights medicines requiring immediate reordering.

5. Module 5: Data Persistence Engine

- Automatically loads inventory records on startup and serializes data to disk upon modifications and exit.

5. Non-Functional Requirements

1. Usability: Simple, self-explanatory menu options (1–5) with guided console prompts and user-friendly error messages.

2. Reliability & Robustness: Implements custom exception handling (`InvalidStockException`) and defensive input parsing to prevent crashes on invalid inputs.

3. Performance & Efficiency: Uses in-memory hashing (`HashMap<String, Medicine>`) offering average $O(1)$ lookup and update times during runtime.

4. Maintainability & Modularity: Clean separation of concerns adhering to standard Java package structures: `models`, `services`, `utils`, and `exceptions`.

6. System Architecture

The application is structured into four distinct logical layers:

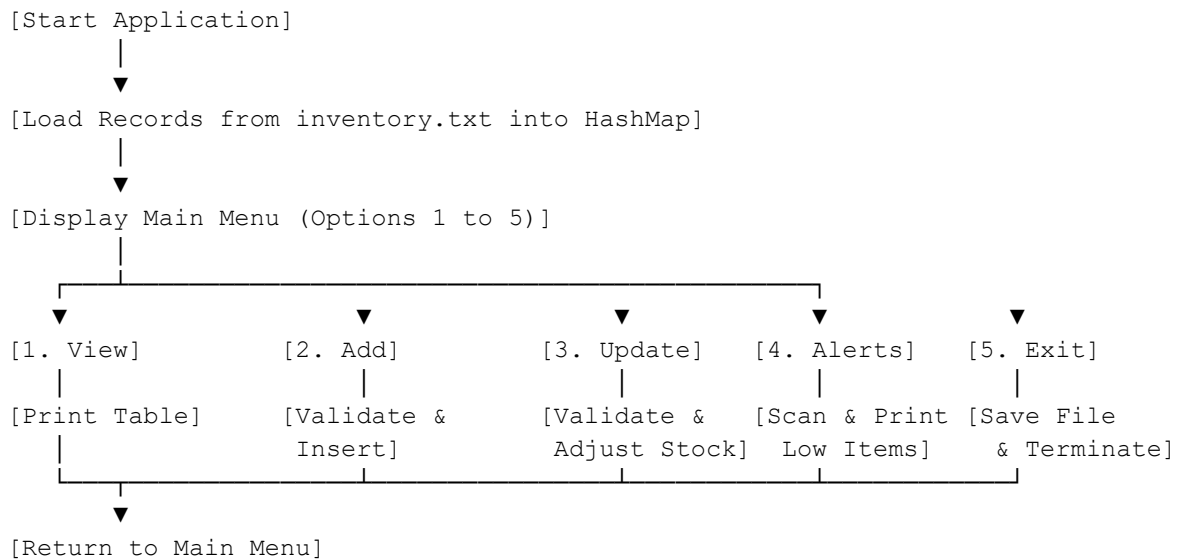
- **Presentation Layer** (`com.medicinetracker.Main`): Handles console UI, menu loop, user input collection, and display formatting.
- **Service Layer** (`com.medicinetracker.services`): Contains business logic (`InventoryService` for stock operations, `AlertService` for threshold warnings).
- **Domain Model Layer** (`com.medicinetracker.models`): Contains the `Medicine` entity encapsulating state and properties.
- **Utility & Data Layer** (`com.medicinetracker.utils`, `com.medicinetracker.exceptions`): Manages file input/output (`FileUtil`), input validation (`ValidationUtil`), and custom domain exceptions (`InvalidStockException`).

7. Design Diagrams

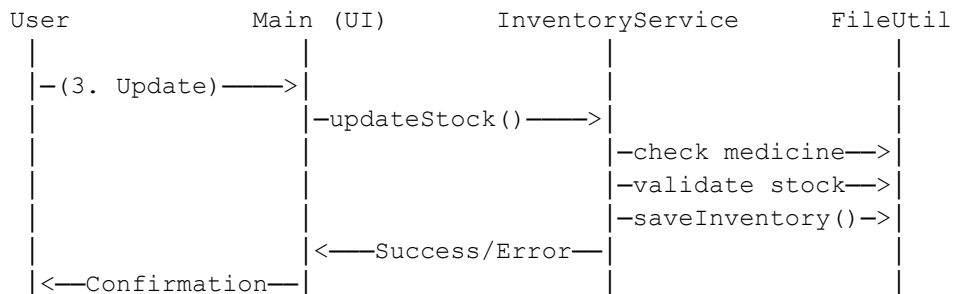
7.1 Use Case Diagram

- **Primary Actor:** Pharmacist / Store Manager
- **Use Cases:**
 1. UC-01: View Current Stock Records
 2. UC-02: Add New Medicine Record
 3. UC-03: Update Existing Stock (Add / Dispense)
 4. UC-04: Trigger Low-Stock Alerts
 5. UC-05: Save and Exit Session

7.2 Process Flow / Workflow Diagram



7.3 Sequence Diagram (Stock Update Flow)



7.4 Class / Component Diagram

- **`Medicine`**

- Fields: name: String, quantity: int, expiryDate: String
- Methods: getName(), getQuantity(), setQuantity(), getExpiryDate(), toString()
- **`InventoryService`**
 - Fields: inventory: Map<String, Medicine>
 - Methods: addMedicine(), updateStock(), displayInventory(), getInventory()
- **`AlertService`**
 - Methods: checkLowStock(Map<String, Medicine>)
- **`ValidationUtil`**
 - Methods: isValidQuantity(int), isValidDate(String)
- **`FileUtil`**
 - Methods: loadInventory(), saveInventory()
- **`InvalidStockException`**
 - Extends java.lang.Exception

7.5 Database / Storage Design (File Schema)

Data is stored persistently in plain text format (inventory.txt) with comma-separated values:

```
<MedicineName>,<Quantity>,<ExpiryDate>
```

Example records:

- Paracetamol,50,12/2026
- Amoxicillin,8,06/2025
- Cetirizine,4,09/2025

8. Design Decisions & Rationale

1. Use of Java Collections (`HashMap`):

- Using a key-value structure with lowercase medicine names as keys allows instant lookups ($O(1)$) and inherently prevents duplicate records.

2. Text-File Persistence:

- Chosen to keep the application self-contained and portable without requiring external database drivers or database server installations.

3. Separation of Concerns (Modular Packages):

- Grouping source files into distinct packages (models, services, utils, exceptions) ensures high cohesion, low coupling, and easy code readability.

4. Custom Exception Handling:

- Defining `InvalidStockException` allows business-logic violations (such as attempting to dispense more units than currently in stock) to be communicated clearly without terminating the JVM.

9. Implementation Details

The project is built entirely in Java (Standard Edition) comprising 7 well-defined classes:

1. `Medicine.java`: Encapsulated domain entity with attributes and getters/setters.
2. `InvalidStockException.java`: Checked exception for domain boundary enforcement.
3. `ValidationUtil.java`: Static validation methods for numerical limits and expiry date formats.
4. `FileUtil.java`: Stream-based file operations utilizing `BufferedReader` and `BufferedWriter`.
5. `InventoryService.java`: Core operational methods implementing business logic and CRUD operations.
6. `AlertService.java`: Business rule engine monitoring minimum stock thresholds.
7. `Main.java`: Interactive driver program managing user sessions and CLI interactions.

10. Screenshots / Execution Results

```
=====
Welcome to Medicine Tracking System
=====
```

Menu Options:

1. View Inventory
2. Add New Medicine
3. Update Medicine Stock
4. Check Low Stock Alerts
5. Exit

Enter your choice: 1

--- Current Inventory ---

Medicine: paracetamol	Quantity: 50	Expiry: 12/2026
Medicine: amoxicillin	Quantity: 8	Expiry: 06/2025
Medicine: cetirizine	Quantity: 4	Expiry: 09/2025

Menu Options:

1. View Inventory
2. Add New Medicine
3. Update Medicine Stock
4. Check Low Stock Alerts
5. Exit

Enter your choice: 4

```
--- Low Stock Alerts (Threshold: < 10) ---  
ALERT: amoxicillin is running low! Current stock: 8  
ALERT: cetirizine is running low! Current stock: 4  
-----
```

11. Testing Approach

Comprehensive testing was conducted covering key functional scenarios:

1. Positive Testing:

- Added valid medicines with valid dates (12/2026) and observed correct persistence.
- Updated stock by positive increments and verified updated totals.

2. Boundary & Negative Testing:

- Attempted to dispense 15 units when only 10 were available; verified that `InvalidStockException` was thrown and handled gracefully.
- Entered negative numbers during new medicine registration; validated rejection by `ValidationUtil`.
- Tested invalid date formats (e.g., 13/2025 or 12-25); validated rejection by date validator.

3. Data Integrity Testing:

- Modified stock, exited application (Option 5), and relaunched; confirmed all changes were faithfully restored from `inventory.txt`.

12. Challenges Faced

1. Scanner Newline Consumption:

- Problem: Reading integers using `nextInt()` left newline tokens in the buffer, causing subsequent `nextLine()` prompts to be skipped.
- Solution: Uniformly read lines using `scanner.nextLine()` and explicitly parsed numbers with `Integer.parseInt()` within a try-catch block.

2. File I/O Graceful Recovery:

- Problem: If `inventory.txt` does not exist on initial run, file reading threw an `IOException`.
- Solution: Added defensive file existence checks; if the file is absent, an empty repository is initialized cleanly.

13. Learnings & Key Takeaways

- Practical implementation of Object-Oriented Programming (OOP) concepts in a real-world scenario.
- Practical experience with Java I/O streams (`BufferedReader`, `BufferedWriter`) and file lifecycle management.
- Designing modular, clean architecture separating domain models, utilities, and application services.
- Writing robust error-handling logic using custom Java exceptions.

14. Future Enhancements

- 1. Relational Database Integration:** Transition from text files to an embedded SQLite or PostgreSQL database using JDBC.
- 2. Graphical User Interface (GUI):** Develop an interactive UI using JavaFX or Swing.
- 3. Expiry Date Notification:** Automatic comparison of expiry dates with system time (`java.time.LocalDate`) to alert staff about expiring medicines.
- 4. Export Capabilities:** Automated export of inventory audits and transaction reports into PDF/Excel formats.

15. References

1. Oracle Java Documentation:
<https://docs.oracle.com/en/java/javase/>
2. Herbert Schildt, *Java: The Complete Reference*, McGraw-Hill Education.
3. VITyarthi Project Guidelines & Evaluation Rubric, VIT Bhopal University.